

Module 50 - Synchronous vs Asynchronous

1. What is Synchronous?

Synchronous execution means tasks are executed one after another in order, and the next task waits until the current task is completed.

2. What is Asynchronous?

Asynchronous execution means tasks can start and continue without waiting for another task to finish, allowing other tasks to execute while waiting.

Example Code:

```
function f1(){
  console.log("First");
}

function f2(){
  console.log("Second");
}

function f3(){
  console.log("Third");
}

f1();           // Synchronous: executes immediately.
setTimeout(f2, 2000); // Asynchronous: executes after at least 2 seconds.
f3();           // Synchronous: executes immediately after f1().

/*
  Call Stack --> Web API --> Call Back Queue --> Event Loop

  Event Loop:

  1. Micro Task Queue - First Priority
    - Promise.then()

  2. Macro Task Queue - Second Priority
    - setTimeout
    - setInterval
*/
```

- **Call Stack** → Executes JavaScript functions one by one.
- **Web API** → Handles time-consuming tasks in the background.
- **Callback Queue** → Holds completed asynchronous callbacks waiting to run.
- **Event Loop** → Checks the Call Stack and moves callbacks to it when it is empty.
- **Microtask Queue** → A queue that stores high-priority asynchronous tasks that run before the callback queue.
- **Macro Task Queue** → A queue that stores asynchronous tasks that run after the microtask queue.